RV-GEMM: Neural Network Inference Acceleration with Near-Memory GEMM Instructions on RISC-V

Xingbo Wang
Southern University of Science and Technology
Shenzhen, China
12231034@mail.sustech.edu.cn

Chenxi Feng
Southern University of Science and Technology
Shenzhen, China
12232148@mail.sustech.edu.cn

Bingzhen Chen
Southern University of Science and Technology
Shenzhen, China
12132103@mail.sustech.edu.cn

Qi Wang
The University of British Columbia
Vancouver, Canada
wq@student.uba.ca

Yucong Huang
Hong Kong University of Science and Technology
Hong Kong, China
yhuangdp@connect.ust.hk

Terry Tao Ye*
Southern University of Science and Technology
Shenzhen, China
yet@sustech.edu.cn

ABSTRACT

General Matrix Multiply (GEMM), as a fundamental operation in neural network, plays an important role in artificial intelligence and signal processing applications. In this paper, we proposed three SMID RISC-V custom instructions to accelerate GEMM computations, supporting multiple precisions including 32-bit, 16-bit and 8-bit fixed. Furthermore, we implemented address calculation and loop control units along with the GEMM acceleration module to reduce the memory access overhead. These three GEMM custom instructions, along with the near-memory optimization units, were incorporated in the RV-GEMM processor and implemented on the FPGA platform for speedup evaluation. It was also compiled in Synopsys Design Compiler with CMOS 55nm process for hardware overhead estimation. Compared to the baseline RISC-V processor, for GEMM computations under precisions of 32-bit, 16-bit and 8-bit fixed, the RV-GEMM processor achieved speedup ratios of 15.8×, 28.7× and 42.5×. The peak energy efficiency also reached 260 GOPS/W, 420 GOPS/W and 609 GOPS/W, respectively.

CCS CONCEPTS

• **Hardware → Hardware accelerators**; • **Computer systems organization → Single instruction, multiple data**.

KEYWORDS

RISC-V, GEMM, Hardware Acceleration, Custom Instruction Sets

ACM Reference Format:
Xingbo Wang, Chenxi Feng, Bingzhen Chen, Qi Wang, Yucong Huang, and Terry Tao Ye. 2024. RV-GEMM: Neural Network Inference Acceleration with Near-Memory GEMM Instructions on RISC-V . In *21st ACM International Conference on Computing Frontiers (CF '24), May 7–9, 2024, Ischia, Italy.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3649153.3649181

*Corresponding author.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
CF '24, May 7–9, 2024, Ischia, Italy
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0597-7/24/05
https://doi.org/10.1145/3649153.3649181

1 INTRODUCTION

RISC-V architecture has demonstrated great potentials in edge computing platforms. Benefit from its simple instruction set, the hardware complexity and power consumption of RISC-V implementation can be greatly reduced. Furthermore, RISC-V takes advantages of its extendable custom instructions that can be tailor-made to accelerate application-specific tasks. Currently, a large variety of edge applications have been implemented based on RISC-V Instruction Set Architecture, such as software defined radio[1], cryptography[2], robotics and other AI-enhanced IOT devices, etc.

General Matrix Multiplication (GEMM), as a fundamental operation in neural networks, accounts for a significant portion of computation loads in various signal processing and AI operations. However, in traditional RISC-V processors, GEMM operations require extensive memory accesses and the execution efficiency is limited by the CPU-memory bandwidth as well as memory READ/WRITE operations.

In this paper, we propose an optimized RISC-V architecture, called RV-GEMM, which aims to accelerate GEMM computations through three SIMD (Single Instruction Multiple Data) RISC-V custom extension instructions. The acceleration is achieved through a dedicated GEMM co-processor, which consists of 16 parallel multiplication units with configurable precision of 32-bit, 16-bit and 8-bit fixed. Additionally, we constructed a loop control unit and an address calculation unit within the co-processor. These two units enable many memory-related operations to be executed between the memory and the GEMM acceleration unit, and significantly reduce the amount of memory accesses from the CPU. For example, when performing matrix multiplication with M=N=K=4 at INT8 precision, the baseline processor requires 128 READ operations, while RV-GEMM require 8 READ operations. With the optimized SIMD custom instructions, RV-GEMM achieved speedup ratios of 15.8×, 28.7× and 42.5× for 32-bit, 16-bit and 8-bit fixed GEMM computations, as compared to the baseline RISC-V processor.

We implemented the proposed RV-GEMM processor on Nexys A7-100T FPGA platform. Additionally, it was also compiled in Synopsys Design Compiler under the CMOS 55nm technology. Experiments demonstrated that the RV-GEMM processor, can significantly reduce inference time and energy consumption for GEMM-related tasks. The computational efficiency reached 260 GOPS/W,

420 GOPS/W and 609 GOPS/W for 32-bit, 16-bit and 8-bit fixed GEMM computations, respectively.

2 RV-GEMM ARCHITECTURE

2.1 Baseline RISC-V Processor

We adopted the RI5CY[3] as the baseline processor. RI5CY is a 32-bit, four-stage pipelined, in-order dispatch RISC-V processor that supports the RV32IMAC instruction set. The four pipeline stages include Instruction Fetch (IF), Instruction Decode (ID), Execute (EX) and Write-Back Memory Access (WB). The processor architecture, along with custom units are highlighted in Fig 1. In the ID stage, we revised the decoder and controller to support the decoding and control of our custom instructions. In the EX stage, a dedicated GEMM acceleration module (GEMM co-processor) was embedded for GEMM operations. In the WB stage, the Load Store Unit (LSU) was modified to accommodate the multi-cycle memory access requirement.

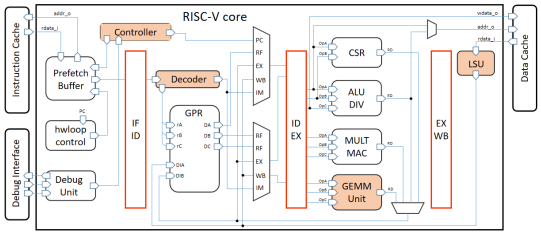


Figure 1: RI5CY Architechture

2.2 GEMM Co-Processor

The structure of the GEMM co-processor is shown in Fig 2. The GEMM computation is performed on PE array, which consists of 4×4 32-bit multiplication units with 2 inputs for each. The GEMM co-processor utilizes 32 32-bit registers that include control registers, data registers and output registers. The control registers store basic GEMM parameters, including precision, matrix memory locations and dimensional parameters. The data registers consist of 8 32-bit registers used to store the data elements of the two matrices. The output registers consist of 16 32-bit registers used to store the accumulated results of GEMM operations. For matrix sizes smaller than 4×4, padding is required before invoking the GEMM co-processor.

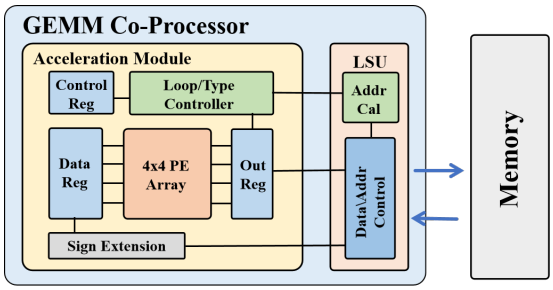


Figure 2: GEMM Co-Processor Architechture

GEMM acceleration can be executed in three stages:

Memory Indexing and Parameter Loading: The control registers are loaded with operational parameters, such as the precision of the operation. The LSU reads 4 data elements from each matrix based on their base addresses. The data loaded is extended with sign bits to fill up to 32 bits using the Sign Extension unit. The computation of GEMM, as shown in Fig 3, needs to load elements of matrices in three dimensions, i.e., M, N, and K. With 16 32-bit multiplication units, 16 (4×4) products can be calculated in parallel.

Each iteration completes 4×4 products in the K dimension. The memory mapping of the two matrices is also arranged such that the elements can be loaded sequentially during the multiplication, i.e., the first matrix is addressed with the M index, and the second matrix is addressed with the N index in the memory respectively. Reading 8 data elements requires 8, 4 and 2 clock cycles for 32-bit, 16-bit and 8- bit precision.

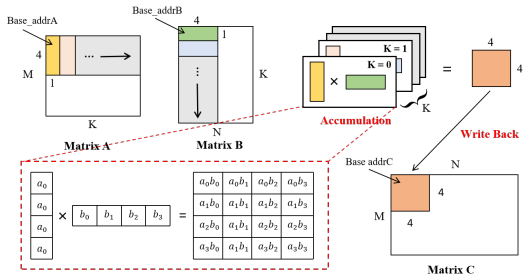


Figure 3: GEMM Calculation Flow

GEMM Calculation: The basic calculation process of matrix multiplication is shown in Fig 3. After the Parameter Loading stage, 4 elements from each matrix are loaded into the PE Array for 16 parallel multiplication operations, resulting in a 4×4 matrix with 16 partial products. The matrix operation is performed incrementally along the K dimension and the partial products accumulated until elements on the K dimension are completed. Then the LSU computes the address offset for the next 4 elements for the Matrix A in M dimension and Matrix B in N dimension. For example, the base addresses used for the first calculation are Base_addrA and Base_addrB. Before the second iteration, the values of address registers are updated to Base_addr+M and Base_addr+N for next matrices read.

Results Write-back: When the iteration ends, the 16 partial products calculated on the K dimension are written back to memory. To avoid data reordering after write-back, the address registers are also updated. Base_addrC+N is used as the address for the next write-back. This ensures that the data remains arranged in an M×N format after the matrix computation is complete.

3 SIMD CUSTOM INSTRUCTION SETS

The proposed GEMM custom instructions use the R-type instruction format of RISC-V. We chose 0x77 as the opcode for the GEMM instructions. Three custom instructions were implemented specifically for GEMM operations, their format is shown in Table 1.

Table 1: SIMD Custom Instructions

Instruction	Funct7	Rs2	Rs1	Funct3	Rd
GEMM_SET	0	Type/K	M/N	0	/
GEMM_CAL	1	A_addr	B_addr	0	/
GEMM_WB	2	WB_addr	/	0	/

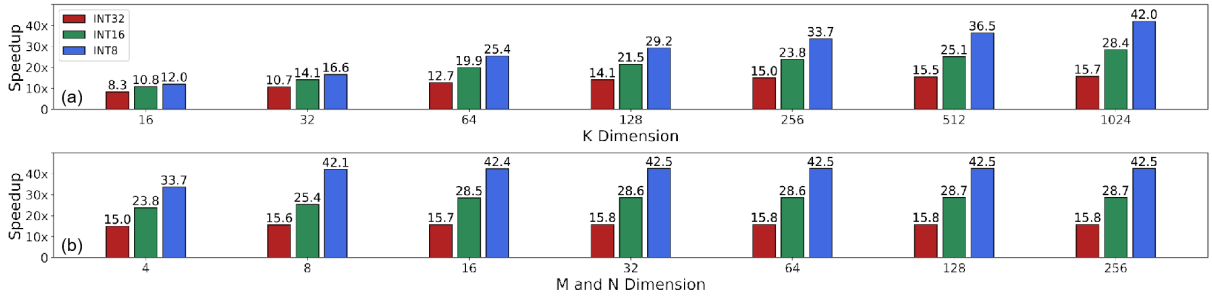


Figure 4: GEMM Speedup with (a) K Dimension, (b) M and N Dimension Increasing

GEMM_SET: This instruction uses two source registers to pass the parameters for the dimensions of M, N and K in the GEMM operation, and also sets the precision levels. The high 16 bits of rs2 are used to set the precision of the operation, while the low 16 bits of rs2 store the K dimension information. The high 16 bits of rs1 store the M dimension, while the low 16 bits store the N dimension.

GEMM_CAL: This instruction takes two source registers, which contain the base addresses of matrices A and B. This instruction invokes the PE Array in conjunction with address calculation and loop control units to perform data reading and computation for the 4×4 matrix multiplication. The execution of this instruction will continue until the partial products are finished accumulation in the K dimension.

GEMM_WB: This instruction is used to write back the 16 results after the GEMM calculation. The rs1 is used to transfer the memory address. Since 16 results are stored in 32-bit registers, the write-back process requires 16 cycles. After every four data write-backs, the address calculation unit upgrades the address register to avoid output reordering.

4 BENCHMARK TESTS AND RESULTS

4.1 GEMM Acceleration

We tested the GEMM acceleration of RV-GEMM with three different arithmetic precision levels: INT32, INT16 and INT8. We used the RV32IM base instruction set without GEMM as the baseline and calculated the speedup achieved when using the RV32IM+GEMM extension instruction set. We performed two types of tests. The first test maintains the matrix M and N dimensions at M=4 and N=4 while increasing the K dimension to evaluate the speedup. The second test maintains the K dimension at K=256 and evaluates the speedup as M and N dimensions increased. The specific speedup results are shown in the Fig 5.

In the first test, the speedup in all three precision modes improves as K increases. This improvement is attributed to the acceleration module, which reduces internal memory accesses and minimizes instruction overhead during iteration loops. Compared to the RV32IM instructions, our SIMD custom instructions fully utilizes the 32-bit bandwidth in INT16 and INT8 precision levels, resulting in more significant acceleration as compared to INT32 precision. When K reaches 1024, tests with three precision levels achieve speedups of 15.7×, 28.4× and 42.0×, respectively.

In the second test, the speedup stabilizes as M and N increase in all three precision levels. The main reason is that the address calculation unit avoids the need of reordering for large matrix inputs and outputs. Similar to the results in the first test, INT8 and INT16 arithmetic precisions outperform INT32 due to their efficient utilization of bandwidth. When both M and N dimensions are fixed at 256, the speedup ratios for the three arithmetic precision levels are 15.8×, 28.7× and 42.5×, respectively. Our custom instructions demonstrate excellent acceleration effects when running GEMM operations with different precision levels and dimensions.

4.2 Neural Network Acceleration

To demonstrate the acceleration effect of RV-GEMM in both simple and complex network structures, we selected two fully connected networks with 784-128-10 neurons and 784-400-10 neurons in each layer respectively, and two convolutional neural networks, i.e., LeNet-5 and VGG11 respectively. All convolution operations were transformed using the IM2COL algorithm to invoke the RV-GEMM acceleration module. We tested these four networks under three different precisions, i.e., INT32, INT16 and INT8.

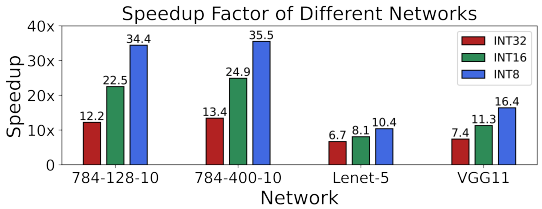


Figure 5: Speedup Evaluation of Different Neural Networks

Fig 5 illustrates the acceleration effect of RV-GEMM on these neural networks. In the two fully connected networks, the 784-400-10 configuration achieved better speedup as compared to the 784-128-10 configuration. This is mainly because the matrix multiplication within has a higher channel depth, resulting in more significant acceleration. The peak acceleration ratios reached 13.4×, 24.9× and 35.5× for the three precisions of INT32, INT16 and INT8 respectively. In the LeNet-5 test, the speedup ratios are 6.7×, 8.1× and 10.4× respectively. The smaller speedup ratios, as comapred to those in the fully-connected networks, are primarily due to the data rearrangement operations involved in IM2COL, which takes up a significant portion of running time. Additionally, extra operations such as pooling and ReLU also have some impacts. VGG11, with its deeper convolutional channels as compared to LeNet-5, achieved higher acceleration ratios of 7.4×, 11.3× and 16.4× for the three precisions of INT32, INT16 and INT8, respectively.

5 IMPLEMENTATION AND RESOURCE CONSUMPTION COMPARISON

5.1 FPGA Implementation

We evaluated the resources and power consumption of the RV-GEMM processor on Nexys A7 100T development board, running at a clock frequency of 50MHz. The Table 2 provides a detailed comparison of resources between the baseline RI5CY processor and the RV-GEMM processor with the custom GEMM instruction. After adding the custom GEMM instructions, the LUT and FF resources are increased by 34% and 36% respectively, as compared to the baseline processor. The internal multiplications are all implemented using DSP modules, resulting in a significant DSP increase of 10.6 times as compared to the baseline processor. The overall power consumption is increased from 0.227W to 0.233W, representing a 2.6% increase.

Table 2: Resources of RI5CY and RV-GEMM on FPGA

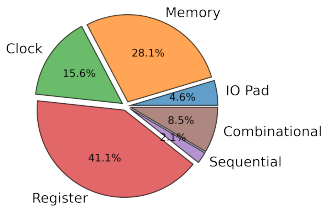
Resources	RI5CY	RV-GEMM	Available
LUT	5452	7303	63400
FF	2632	3579	126800
BRAM	128	128	135
DSP	5	53	240

5.2 ASIC Implementation

We also compiled the RV-GEMM processor in Synopsys Design Compiler in CMOS 55nm process. The Fig 7 shows the basic parameters and specific power composition of RV-GEMM after synthesis. The RV-GEMM architecture operates at a frequency of 100MHz, occupying an area of 3.0mm^2. We equipped RV-GEMM with 32KB of instruction memory and 512KB of data memory on-chip. The operating voltage of RV-GEMM is 1.2V and the power consumption is 1.06mW. In terms of power consumption breakdown, the internal registers account for 41.1%, memory accounts for 28.1%, the clock accounts for 15.6%, combinational logic accounts for 8.5% and the IO pad and sequential components account for 8.5% and 2.1%, respectively. Under three different precisionsm, i.e., INT32, INT16 and INT8, the peak GEMM energy efficiency reaches 260 GOPS/W, 420 GOPS/W and 609 GOPS/W, respectively.

Design	RV-GEMM
Process	CMOS 55nm
CPU	RISC-V
Frequency	100 MHz
Area	3.0mm^2
RAM	32 KB + 512 KB
Voltage	1.2 V
Power	1.06 mW

(a)



(b)

Figure 6: RV-GEMM Parameters

5.3 Comparison With Other Processor

We compared RV-GEMM with three other processors. Table 3 presents the specific parameters of RV-GEMM and other processors. Compared to Eyeriss[4] and FNT[5], RV-GEMM provides more comprehensive support for different arithmetic precisions. Additionally, the RISC-V-based design of RV-GEMM allows it to be used not only for accelerating DNNs but also for executing general tasks. Under 16-bit precision, RV-GEMM achieves energy efficiency improvements of 2.27× compared to Eyeriss and 1.17× compared to FNT. Dustin[6], also based on RISC-V design, supports a wider range of arithmetic precisions. RV-GEMM achieves 2.0×, 3.11× and 6.5× energy efficiency improvements compared to Dustin under 8-bit, 16-bit and 32-bit precisions.

Table 3: Comparison with prior work

Design	Eyeriss[4]	FNT[5]	Dustin[6]	**This work**
Process	65nm	65nm	65nm	55nm
Area (mm^2)	12.25	11.47	10.0	3.0
CPU	-	-	RISC-V	RISC-V
Power (mW)	278	805	156	1.06
Int Precision	16-bit	16-bit	2,4,8, 16,32-bit	8,16,32-bit
Clock (MHz)	200	200	200	100
Voltage	0.82-1.17 V	1.2 V	0.8-1.2 V	1.2 V
Peak Energy Efficiency (GOPS/W)	184.9	357.8	303(8b) 135(16b) 40(32b)	609(8b) 420(16b) 260(32b)

6 CONCLUSION

In this paper, we proposed the RV-GEMM processor, which incorporates three high-efficient, RISC-V based, GEMM acceleration instructions with configurable precisions (INT32, INT16 and INT8). Consisting of 16 parallel multiplication PEs, a loop control unit and an address calculation unit, the RV-GEMM processor greatly accelerates the GEMM computation while reducing the memory access overhead. Through benchmark tests, the RV-GEMM processor exhibits significant speedups in neural network computation under INT32, INT16 and INT8 precision levels.

REFERENCES

[1] Cecil Accetti RA Melo and Edna Barros. 2016. Oolong: a baseband processor extension to the risc-v isa. In *2016 IEEE 27th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE, 241–242.

[2] Guozhu Xin, Jun Han, Tianyu Yin, Yuchao Zhou, Jianwei Yang, Xu Cheng, and Xiaoyang Zeng. 2020. Vpqc: a domain-specific vector processor for post-quantum cryptography based on risc-v architecture. *IEEE transactions on circuits and systems I: regular papers*, 67, 8, 2672–2684.

[3] Michael Gautschi, Pasquale Davide Schiavone, Andreas Traber, Igor Loi, Antonio Pullini, Davide Rossi, Eric Flamand, Frank K Gürkaynak, and Luca Benini. 2017. Near-threshold risc-v core with dsp extensions for scalable iot endpoint devices. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 25, 10, 2700–2713.

[4] Yu-Hsin Chen, Tushar Krishna, Joel S Emer, and Vivienne Sze. 2016. Eyeriss: an energy-efficient reconfigurable accelerator for deep convolutional neural networks. *IEEE journal of solid-state circuits*, 52, 1, 127–138.

[5] Weihong Xu, Zaichen Zhang, Xiaohu You, and Chuan Zhang. 2020. Reconfigurable and low-complexity accelerator for convolutional and generative networks over finite fields. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 39, 12, 4894–4907.

[6] Angelo Garofalo, Gianmarco Ottavi, Alfio Di Mauro, Francesco Conti, Giuseppe Tagliavini, Luca Benini, and Davide Rossi. 2021. A 1.15 tops/w, 16-cores parallel ultra-low power cluster with 2b-to-32b fully flexible bit-precision and vector lockstep execution mode. In *ESSCIRC 2021-IEEE 47th European Solid State Circuits Conference (ESSCIRC)*. IEEE, 267–270.